

servo_monitor.py — Complete Walkthrough

A section-by-section, line-by-line explanation of the Servo Monitor GUI. Written so you can explain any part of it to a colleague, and so you know *why* each odd-looking piece exists.

1. Big Picture

The program is one file with two halves:

The protocol half (lines 81-406) — three functions that talk to the servo over the DPC-CAN USB adapter's proprietary serial protocol (which we reverse-engineered from USB captures of the official Hitec app):

Function	What it does
<code>read_all_data()</code>	Reads ID1, ID2 and all ten editable settings from the live servo
<code>read_ids_only()</code>	Same wake-up, but reads only ID1/ID2 (fast — used by the Read ID button)
<code>write_all(values)</code>	Writes a dict of <code>{register -> value}</code> , issues one save command, reads everything back to confirm

The GUI half (lines 413-775) — `main()` builds a tkinter window: an ID display with a Read ID button, a profile dropdown backed by `servo_profiles.json`, entry fields and dropdowns for the ten settings, two save buttons ("to profile only" / "to servo only"), a status line, and a once-per-second `poll()` loop that watches for the USB adapter appearing/disappearing.

Key architectural decision: **all serial I/O happens on the tkinter main thread.** No `threading` module anywhere. On this system, background threads broke pyserial's timing, so instead the GUI uses `root.after(1000, poll)` for periodic work, and the buttons simply block briefly while a read/write runs (with `root.update()` called first so the status message paints before the blocking starts).

2. Imports and Constants (lines 10-63)

```
python
```

```
import json, os, sys, time
import tkinter as tk
from tkinter import ttk
import serial
import serial.tools.list_ports
```

- `json` — the profile file is JSON.
- `os`/`sys` — building the profile file path, and detecting a PyInstaller build.
- `time` — every timing delay in the protocol (`time.sleep`) and every read timeout (`time.time()` deadlines).
- `tkinter as tk` — the GUI toolkit. `ttk` is imported separately only for `ttk.Combobox` (the profile dropdown), which classic tk doesn't have.
- `serial` — pyserial, for the COM port. `serial.tools.list_ports` is the sub-module that enumerates connected USB serial devices with their vendor/product IDs.

```
python
```

```
TARGET_VID = 0x10C4
TARGET_PID = 0xEA60
```

Every USB device advertises a Vendor ID and Product ID. `0x10C4` is Silicon Labs, `0xEA60` is their CP210x USB-to-UART bridge chip — the chip inside the DPC-CAN. Filtering on this pair means the app finds the adapter automatically, no matter which COM number Windows assigned it, and never accidentally opens some other serial device.

```
python
```

```
BAUD = 115200
```

The speed of the **USB serial link between PC and DPC-CAN**. Not to be confused with the CAN bus baud rate (the "Baud Rate [kbps]" setting in the GUI, register `0x38`) — that's the speed of the wire between the DPC-CAN and the servo, and is a servo setting, not a PC setting.

PyInstaller path detection (lines 41–46)

```
python
```

```
if getattr(sys, "frozen", False):
    APP_DIR = os.path.dirname(os.path.abspath(sys.executable))
else:
    APP_DIR = os.path.dirname(os.path.abspath(__file__))

PROFILES_FILE = os.path.join(APP_DIR, "servo_profiles.json")
```

The problem this solves: when you build the app into a single `.exe` with PyInstaller (`--onefile`), the exe silently unpacks itself into a **temporary folder** at startup, and `__file__` points inside that temp folder — which is deleted when the app closes. So if you saved `servo_profiles.json` next to `__file__`, your profiles would vanish every session.

PyInstaller sets `sys.frozen = True` in its builds. `getattr(sys, "frozen", False)` reads that flag safely (returns `False` when the attribute doesn't exist, i.e. when running as a plain script). In a frozen build we use `sys.executable` — the real, permanent location of the `.exe` — so the JSON file lives next to the exe and survives.

Default profile (lines 50–63)

`DEFAULT_PROFILE_SETTINGS` is what a brand-new profile is seeded with — the manual's documented factory defaults for each register (e.g. `pos_max` 10922, `pos_mid` 8192, `id1` 1). When you click **Add** with a new name, this dict is copied into the profile.

3. Profile Persistence (lines 66–78)

```
python

def load_profiles():
    if not os.path.exists(PROFILES_FILE):
        return {}
    with open(PROFILES_FILE, "r") as f:
        return json.load(f)

def save_profiles(profiles):
    with open(PROFILES_FILE, "w") as f:
        json.dump(profiles, f, indent=2)
```

The on-disk format is a single JSON object: `{"profile name": {settings dict}, ...}`. `load_profiles` returns `{}` on first run instead of crashing on a missing file. `indent=2` just makes the JSON human-readable so you can inspect/edit it by hand.

This block appears **three times** (in `read_all_data`, `read_ids_only`, and `write_all`) because each function opens the port fresh, and the servo/adaptor must be woken up every time a session starts. It is a byte-for-byte replay of what the official Hitec app sends, discovered via USB capture. We don't know what every byte "means" — we know the sequence works, and departures from it don't. That's the honest answer if anyone asks.

Step by step (using `read_all_data`'s copy, lines 96–138):

```
python

ser = serial.Serial(port, BAUD, timeout=0.1)
time.sleep(0.2)
```

Opens the COM port with a **0.1 s read timeout** — meaning every `ser.read(256)` call later returns after at most 0.1 s even if no data arrived (returns whatever it got, possibly `b""`). This non-blocking-ish behavior is what makes the read loops responsive. The 0.2 s sleep gives Windows/the CP210x driver a moment to settle after opening.

```
python

seq = 0xc0
```

Initializes the rolling **sequence byte** used in the probe frames below. Frames carry a sequence number the adapter echoes/tracks; it starts at `0xc0` here to mirror the captured session.

```
python

for _ in range(5):
    ser.write(bytes([0x02, 0x53, 0x00, 0xff, 0x52, 0x03, 0x03]))
    time.sleep(0.03)
```

Five repeats of a short `0x53` ('S') frame — the first thing the official app transmits. Framing convention throughout: `0x02` (STX, start-of-text) opens a frame, `0x03` (ETX, end-of-text) closes it.

```
python

for _ in range(11):
    ser.write(bytes([0x02, 0x58, 0x58, 0x03]))
    time.sleep(0.03)
```

Eleven repeats of an `(0x58)` ('X') frame — likely a reset/flush of the adapter's state.

```
python
```

```
ser.write(bytes([0x3a, 0x41, 0x3a, 0x41, 0x3a, 0x41]))
```

The odd one out: `(:A:A:A)` in ASCII, no STX/ETX framing. Looks like a mode/attention string for the adapter itself rather than a servo frame.

```
python
```

```
ser.write(bytes([0x02, 0x56, 0x53, 0x00, 0x00, 0x00, 0x00, 0x03]))
time.sleep(0.15)
ser.reset_input_buffer()
```

A `(VS)` (version?) query, then `(reset_input_buffer())` throws away whatever the adapter replied — we don't need the reply, we just need the adapter to have gone through the exchange. You'll see `(reset_input_buffer())` used repeatedly as a "clear the deck" step so stale bytes can't be mistaken for the reply we actually want.

The four activation levels (lines 114–135)

```
python
```

```
for n in range(4):
    activate = bytes([0x02, 0x53, n, 0x01, (n + 0x54) & 0xFF, 0x03, 0x03])
    for _ in range(5):
        ser.write(activate)
        time.sleep(0.03)
```

The captured app performs the scan at four "levels" $N = 0..3$. Byte 2 of the activation frame is N itself, and byte 4 is `(N + 0x54)` — which is the frame's checksum tracking the changing N (so the frame stays internally valid at each level). Each activation is sent 5 times.

Then, at each level, the app probes three "version" registers — `(0xfc)`, `(0xfe)`, `(0x74)` — using **three distinct frame formats** (we called them A, B, C during the reverse engineering):

Format A — the standard register-read frame, sent twice:

```
python
```

```
body = [register & 0xFF, 0x00, 0x00, seq & 0xFF]
hdr = [0x02, 0x42, 0x00, 0xf2, 0x03, 0x80, 0x80]
chk = sum(hdr[1:] + body) & 0xFF
ser.write(bytes(hdr + body + [chk, 0x0a, 0x03]))
seq = (seq + 1) & 0xFF
```

This is the same frame shape used for all real register reads later, so it's worth memorizing:

```
02 42 00 f2 03 80 80 <reg> 00 00 <seq> <checksum> 0a 03
└STX┐          ┌─── body ──┐          └tail┘
```

- Checksum = sum of every byte **after the STX** through the end of the body, masked to one byte (`(& 0xFF)`). The STX itself is excluded (`(hdr[1:])`).
- `seq = (seq + 1) & 0xFF` increments the rolling sequence and wraps it at 255.

Formats B and C are two other probe shapes from the capture. Their tail bytes differ per register, so the code stores them in small lookup dicts:

```
python

tailB = {0xfc: 0x50, 0xfe: 0x54, 0x74: 0x40}[register]
...
tailC = {0xfc: [0xcf, 0x08, 0x03], 0xfe: [0xd3, 0x08, 0x03], 0x74: [0xbf, 0x08, 0x03]}
```

These constants come straight from the capture. (If you check, each tail is consistent with a checksum over that frame's contents — but we recorded them as literals rather than deriving them, since they never change.)

Finally:

```
python

time.sleep(0.3)
ser.reset_input_buffer()
```

Let all scan replies arrive, then discard them all. The port is now "awake" and clean, ready for real reads.

5. Reading a Register — `read_one()` (lines 143–167)

Defined as a nested function inside each protocol function (so it can close over `ser`).

python

```
marker = bytes([0xf3, 0x03, 0x88])
header = [0x02, 0x42, 0x00, 0xf2, 0x03, 0x80, 0x80]
```

`marker` is the key discovery for parsing replies. A register-read reply looks like:

```
04 xx f3 03 88 xx <reg> <lo> <hi> <chk> 08 05
      └──┬──┘
      stable marker
```

The bytes before and around it vary, but `f3 03 88` always appears at a fixed offset before the register number and its 16-bit value. So instead of trying to fully parse the adapter's reply framing (which we never completely decoded), we simply **search the byte stream for the marker** and read fixed offsets after it. Robust and simple.

python

```
def read_one(register, frame_seq):
    body = [register & 0xFF, 0x00, 0x00, frame_seq & 0xFF]
    checksum = sum(header[1:] + body) & 0xFF
    frame = bytes(header + body + [checksum, 0x0a, 0x03])

    ser.reset_input_buffer()
    ser.write(frame)
```

Builds a Format-A read frame for the requested register with the *caller-supplied* sequence number (more on why that matters below), clears any leftover bytes, and sends it.

python

```
collected = b""
t0 = time.time()
value = None
while time.time() - t0 < 3.0:
    chunk = ser.read(256)
    if chunk:
        collected += chunk
```

A 3-second deadline loop. Each `ser.read(256)` waits at most 0.1 s (the port timeout) and appends whatever arrived to a growing `collected` buffer. Accumulating is essential because the reply can arrive **split across multiple reads** — the marker might straddle a chunk boundary, so we always search the whole buffer.

python

```
idx = collected.find(marker)
while idx != -1 and idx + 7 <= len(collected):
    if collected[idx + 4] == register:
        lo, hi = collected[idx + 5], collected[idx + 6]
        value = lo | (hi << 8)
        break
    idx = collected.find(marker, idx + 1)
```

Finds every occurrence of the marker. For each hit, checks that the byte at **marker + 4** is the register we asked for (replies to *other* registers, or stale replies, can be in the stream — this filter is what makes the read reliable). `(idx + 7 <= len(collected))` guards against reading past the buffer when the marker arrived but the value bytes haven't yet.

The value itself is at marker + 5 (low byte) and marker + 6 (high byte); `(lo | (hi << 8))` reassembles the **little-endian 16-bit** value. If this marker hit is for the wrong register, `(find(marker, idx + 1))` moves on to the next occurrence.

Returns the value, or `(None)` if the deadline expired — a `(None)` propagates up and shows as "read failed" / "??" in the GUI instead of crashing.

The sequence-number trap (critical to remember)

Look at how `(read_one)` is called:

python

```
id1 = read_one(0x32, 0xd2)
id2 = read_one(0x3E, 0xd3)
speed = read_one(0x54, 0xd4)
...
speed_dn = read_one(0xDE, 0xdd)
```

The sequence numbers are hard-coded starting at `(0xd2)`. This was the root cause of several mysterious "no reply" failures during development: **the adapter/servo silently ignores frames carrying fresh, never-used sequence numbers** (e.g. `(0xe0)` and up), while numbers in the neighborhood the wake-up sequence itself used (`(0xd2-0xd7)` and nearby) reliably get replies. We never found documentation for why — it's an empirical fact of this adapter. Any new read you add should reuse this range, not invent a new counter.

Signed conversion (lines 185–188)

python

```
def to_signed16(v):
    if v is None:
        return None
    return v - 0x10000 if v >= 0x8000 else v
```

Registers come back as raw unsigned 16-bit values (0–65535). The position registers (Pos Max/Min/Mid) are actually **signed** 16-bit (two's complement, range –32768..32767): a negative position like –5462 is stored on the wire as $65536 - 5462 = 60074$, i.e. any value $\geq 0x8000$ (32768) is really negative. Subtracting $0x10000$ (65536) converts it back. Only the three `(pos_*)` values get this treatment in `read_all_data`; IDs, speed, etc. are genuinely unsigned.

`read_all_data` then packs everything into a dict keyed by the same names used in profiles and the GUI (`"id1"`, `"speed"`, `"pos_max"`, ...) and returns `(data, None)` — the `(result, error)` tuple convention used by all three protocol functions, so callers do `if error: show it` instead of `try/except`.

6. `read_ids_only()` (lines 207–282)

Identical wake-up + identical `read_one`, but it stops after ID1 (`0x32`) and ID2 (`0x3E`). It exists because the full 12-register read takes noticeably longer, and the **Read ID** button only needs to answer "which physical servo is plugged in right now?" — IDs are the one thing that must come from the live servo rather than from a saved profile, because they're the servo's identity.

(Yes, the wake-up code is duplicated three times rather than factored into a helper. That was deliberate during development: each function was proven working end-to-end against the capture, and refactoring shared serial code risks subtle timing changes. A future cleanup could extract `wake_up(ser)` once everything is stable.)

7. `write_all(values)` (lines 285–406)

Takes `values`: a dict of `register -> value`. One port open, one wake-up, then:

The write loop (lines 335–344)

```
python
```

```
wseq = 0xd4
for register, value in values.items():
    u16 = value & 0xFFFF
    lo, hi = u16 & 0xFF, (u16 >> 8) & 0xFF
    whdr = [0x02, 0x42, 0x00, 0xf2, 0x03, 0x80, 0x00]
    wbody = [register & 0xFF, lo, hi, wseq & 0xFF]
    wchk = sum(whdr[1:] + wbody) & 0xFF
    ser.write(bytes(whdr + wbody + [wchk, 0x0a, 0x03]))
    wseq = (wseq + 1) & 0xFF
    time.sleep(0.15)
```

The write frame (confirmed from a real settings-change capture):

```
02 42 00 f2 03 80 00 <addr> <lo> <hi> <seq> <chk> 0a 03
```

Compare with the read frame: **byte 6 is `0x00` for a write vs `0x80` for a read** — that single byte is the read/write flag. The value goes in as little-endian 16-bit; `value & 0xFFFF` handles negative inputs correctly (Python's `&` on a negative int yields the two's complement pattern, so `-5462` becomes `60074` → `lo=0xAA, hi=0xEA`). The write sequence counter also starts in the proven range (`0xd4`). 0.15 s between writes gives the servo time to process each one.

The save command (lines 346–352)

```
python

sbody = [0x70, 0xff, 0xff, wseq & 0xFF]
```

Writing `0xFFFF` to register `0x70` (REG_CONFIG_SAVE) is the "commit to non-volatile memory" command — same write-frame format, magic value. Without this, changed settings would revert on power cycle. It's sent **once at the end**, not per register.

The settle drain (lines 359–362)

```
python

settle_until = time.time() + 1.5
while time.time() < settle_until:
    ser.read(256)
ser.reset_input_buffer()
```

This looks like it should be `time.sleep(1.5)`, and that's exactly what it must **not** be. Discovered the hard way: after a burst of writes the adapter has a backlog of reply traffic,

and if you sleep blind, the USB pipe stalls and the backlog gets *worse* — subsequent reads then time out. Actively calling `ser.read(256)` in a loop keeps draining the pipe (each call blocks ≤ 0.1 s thanks to the port timeout), throwing the data away, so by the end the line is genuinely quiet.

Read-back confirmation (lines 364–406)

```
python
rseq = 0xd2
```

The most important comment in the file lives here. After all the writes and the save, `wseq` has counted well past the proven-good sequence range — and fresh sequence numbers get **silently ignored** (the ID1/ID2 bug). So the confirmation reads deliberately **reset the counter back to** `0xd2` rather than continuing from `wseq`.

The loop then re-reads every register that was written, using the same marker-scan logic as `read_one` (inlined here, with a shorter 1.5 s deadline per register), applies the same signed-16 conversion, and builds `confirmed[register] = value_back` — with `None` for any register whose read-back failed. The GUI uses this dict to show what *actually* stuck.

Note: ID changes always "fail" confirmation in a sense — per the manual, new IDs only take effect after a power cycle, so reading them back immediately may return the old values. That's why the GUI's success message tells you to unplug/replug and press Read ID.

8. The GUI — `main()` (lines 413 onward)

Window and status line

```
python
root = tk.Tk()
root.title("Servo Monitor")
root.geometry("420x800")
status_var = tk.StringVar(value="Waiting for servo...")
```

`tk.StringVar` is tkinter's observable string: any `Label` whose `textvariable` is bound to it updates automatically when you call `.set()`. The status line at the bottom is the app's single feedback channel — every handler ends by setting `status_var`.

ID display + Read ID button (lines 425–445)

Two `StringVars` (`id1_var`, `id2_var`) shown as labels, plus:

python

```
def on_read_id():
    status_var.set("Reading ID...")
    root.update()
    ids, error = read_ids_only()
    ...
```

`root.update()` is a small but important trick: `read_ids_only()` **blocks the main thread** for a few seconds (wake-up + two reads). Without `update()`, tkinter wouldn't get a chance to repaint, so "Reading ID..." would never appear — the window would just look frozen. Calling `update()` forces one repaint *before* the blocking call, so the user sees what's happening. (This is the accepted trade-off of the no-threading design.)

Profile selector (lines 452-467)

- `profiles = load_profiles()` — the in-memory dict, loaded once at startup and kept in sync with disk by `save_profiles()` after every change.
- `ttk.Combobox(..., state="readonly")` — readonly means the user can only *pick* from the list, not type arbitrary text into it; new names go through the separate Entry + Add button instead.
- The Add button uses `command=lambda: on_add_profile()` — a lambda because `on_add_profile` is defined *later* in the function; the lambda defers the name lookup until click time, when it exists. (Direct `command=on_add_profile` here would raise a `NameError` at build time.)

Field definitions (lines 476-504)

`FIELD_INFO` is the single source of truth for the seven plain numeric fields — each tuple is `(profile_key, label_text, register_address)`. Adding a new numeric setting to the app is one line here; the row-building loop, `gather_and_validate`, save, and profile-load code all iterate over the same structures.

The three dropdown settings each get a pair of dicts:

python

```
SAMPLE_POINT_LABELS = {0: "50", 1: "87.5"}
SAMPLE_POINT_VALUES = {label: raw for raw, label in SAMPLE_POINT_LABELS.items()}
```

`LABELS` maps raw register value → human text; `VALUES` is the same mapping inverted (built with a dict comprehension) for going the other way when saving. Defining one from the other guarantees they can never disagree.

The `(entries)` dict maps `(profile_key -> (Entry widget, register))` for the plain fields; `(id_entries)` maps label $\rightarrow$ Entry for ID1/ID2, and `(ID_WRITE_INFO)` carries their validation ranges (ID1: 1-127, ID2: 0-127 — DroneCAN node ID ranges).

Layout notes

Two geometry managers are used, correctly and separately: `(.pack())` stacks the big vertical sections of the window, while `(.grid())` arranges label/entry pairs in aligned rows *inside* `(fields_frame)` and `(profile_frame)`. (Mixing pack and grid inside the *same* container is illegal in tkinter; using them in different containers, as here, is standard practice.) `(fields_frame.columnconfigure(1, minsize=100))` reserves width for the entry column so rows line up; `(sticky="ew")` makes widgets stretch to fill their cell east-west.

`(gather_and_validate())` (lines 560–608)

The shared front-half of both save buttons. It walks every input and produces **two views of the same data**:

- `(to_write): (register -> value)` — what `(write_all())` needs.
- `(settings): (profile_key -> value)` — what the JSON profile stores.

Validation is fail-fast with a three-value return `((to_write, settings, error))`: the moment anything is invalid (non-integer text, an unselected dropdown, an ID out of range) it returns `(None, None, message)` and the caller aborts with the message in the status line — guaranteeing "nothing saved anywhere" on any error, rather than a partial write.

Detail worth knowing: `(int(raw))` on `(entry.get().strip())` rejects anything that isn't a whole number (including the placeholder "-"), and `(settings["id1" if label.startswith("ID1") else "id2"] = value)` maps the display label back to the profile key.

`(on_save_to_servo())` (lines 610–655)

1. `(gather_and_validate())` — bail on error.
2. Set status + `(root.update())` (same repaint-before-blocking trick).
3. `(write_all(to_write))` — the long blocking call.
4. **Overwrite every field with what the servo reported back** (`(confirmed)`), not with what the user typed. This is deliberate: the fields become a display of servo truth. Any register that failed confirmation shows `(??)` and is collected in `(failed)`, and the status message lists them — with the reminder that IDs need an unplug/replug.

For the dropdowns, `(SAMPLE_POINT_LABELS.get(sp_back, ...))` translates the raw value back to its label, falling back to `("?")` (if `(None)`) or the raw number as a string (if it's some unexpected value not in the table).

`on_save_to_profile()` (lines 657–673)

The mirror image: validates the same way, then writes `settings` into `profiles[profile_name]` and saves the JSON. **Zero serial communication** — the servo is untouched. It refuses if no real profile is selected (`"-"` is the "nothing selected" sentinel throughout the app).

Having two explicit buttons instead of one "Save" was a deliberate UX decision: it makes "update my saved template" and "program the physical servo" impossible to confuse.

`fill_settings_fields()` / `on_profile_selected()` (lines 691–724)

Selecting a profile loads its **saved** values into the fields — no live read. Loading is: `entry.delete(0, tk.END)` then `entry.insert(0, ...)` (the standard tkinter idiom for replacing an Entry's text — there is no `entry.set()`), and label-lookup for the dropdowns. Missing keys show `"-"`.

Known cosmetic bug to fix: line 723–724 passes `font=10` to `status_var.set(...)`. `StringVar.set()` takes only the string; the extra keyword will raise a `TypeError` when a profile is selected. Delete `, font=10`.

`on_add_profile()` (lines 726–737)

Reads the name Entry; if the name is new, seeds it with a **copy** of the defaults — `dict(DEFAULT_PROFILE_SETTINGS)` — and this copy matters: without it, every profile would share the *same* dict object, and editing one would silently edit them all. Then persists, refreshes the Combobox's value list, selects the new name, clears the Entry, and calls `on_profile_selected()` to load the defaults into the fields.

```
python
profile_box.bind("<<ComboboxSelected>>", on_profile_selected)
```

`<<ComboboxSelected>>` is a ttk *virtual event* fired when the user picks an item; the handler takes `_event=None` so it can also be called directly (as `on_add_profile` does).

The `poll()` loop (lines 741–770)

```
python
state = {"connected_port": None}
```

Mutable state shared with the nested `poll()` function. It's a dict rather than a plain variable because a nested function can *mutate* an outer dict freely, whereas *rebinding* an outer variable would need a `nonlocal` declaration — the dict is the simpler idiom.

`poll()` runs once a second and only does a **USB presence check** (`find_port()` enumerates ports — no serial traffic at all, so it's cheap and can't disturb the servo):

- **Adapter gone, was connected** → clear every field to `"-"`, reset the profile selector, status "Waiting for servo...". This prevents stale values from a previous servo being mistaken for the newly plugged one.
- **Adapter appeared on a new port** → record it and prompt the user to pick which profile this servo is. Deliberately does **not** auto-read the servo: reading takes seconds and would block the UI right at plug-in; the user chooses when to hit Read ID.
- Reschedules itself: `root.after(1000, poll)` — tkinter's timer mechanism, and the cornerstone of the no-threading design: it runs `poll` on the main thread after ~1000 ms, in between processing GUI events.

```
python

poll()
root.mainloop()
```

Kick off the first poll (which schedules all subsequent ones), then enter tkinter's event loop — `mainloop()` blocks here for the life of the window, dispatching clicks, repaints, and `after` callbacks.

```
python

if __name__ == "__main__":
    main()
```

Standard guard: `main()` runs when the file is executed directly, but not if some other script imports it (e.g. to reuse `read_all_data`).

9. Cheat Sheet — the facts people will ask about

Frame formats

Purpose	Bytes
Register READ request	02 42 00 f2 03 80 80 <reg> 00 00 <seq> <chk> 0a 03
Register WRITE request	02 42 00 f2 03 80 00 <reg> <lo> <hi> <seq> <chk> 0a 03
READ reply (scan for marker)	... f3 03 88 xx <reg> <lo> <hi> <chk> 08 05
Save/commit	write 0xFFFF to register 0x70

- Read vs write differ by one byte: 0x80 vs 0x00 at position 6.
- Checksum = $\text{sum}(\text{all bytes after STX, through body}) \& 0xFF$.
- Values are little-endian 16-bit; positions are signed (two's complement).

Register map used by this app

Register	Setting
0x32	ID1 (Actuator ID, 1-127)
0x3E	ID2 (CAN Node ID, 0-127)
0x54	Speed (0-4095)
0xB0 / 0xB2 / 0xC2	Pos Max / Pos Min / Pos Mid (signed)
0x2E	Stream Time (0-12000)
0xDC / 0xDE	Time Speed Up / Down (ms)
0x40	Sample Point (0=50%, 1=87.5%)
0x38	CAN Baud Rate (0=1000 kbps ... 8=125 kbps)
0x30	Stream Mode (0=Off, 1=On)
0x70	Config save (write 0xFFFF)

The four non-obvious rules

1. **Sequence numbers:** only the proven neighborhood (`0xd2-0xd7`)-ish, seeded from the wake-up counter) reliably gets replies; fresh numbers are silently ignored. Always reset into this range for reads (see the `rseq = 0xd2` comment in `write_all`).
2. **Wake-up is mandatory** every time the port is opened, byte-for-byte as captured.
3. **Never sleep blind after a write burst** — actively read-and-discard to drain the pipe, or subsequent reads stall.
4. **No threads** — all serial on the tkinter main thread, `root.after()` for polling, `root.update()` before any blocking call so the status message paints.

Known issue to fix: `status_var.set(..., font=10)` at line ~724 will raise `TypeError` on profile selection — remove the `font=10` argument.